Actividad 09/17
Bloque 1

Misión 1:

Escenario P1: ¿Morirá en el siguiente frame?
Variable objetivo (Y): is_crash_next_frame. Tipo: Binaria (1 = Choca en el siguiente tick, 0 = Sobrevive).
Variables de entrada (X):

1. distancia_proximo_obstaculo (Numérica): Distancia en píxeles. Básica para saber si ya estamos encima del problema.
2. velocidad_juego (Numérica): Como el juego acelera, la ventana de ms para reaccionar se hace más chica.
3. tipo_obstaculo (Categórica): cactus_chico, cactus_grande, pajaro. Define la caja de colisión (hitbox).
4. posicion_y_dino (Numérica): Coordenada vertical. Nos dice si el dino va subiendo, en el punto más alto, o cayendo.
5. ancho_obstaculo (Numérica): Saltar 3 cactus juntos requiere más tiempo en el aire que saltar uno solo.

Granularidad: Un frame por fila (aprox. cada 16 ms).
Tamaño mínimo razonable: Unos 500,000 frames (unas cuantas horas de juego). Necesitamos un dataset gigante porque los eventos de muerte son anomalías (de miles de frames en una partida, solo 1 es el choque).
Riesgo si está mal definido: Desbalanceo extremo de clases. Si le pasamos los datos crudos, el modelo va a notar que el 99.9% de las veces no pasa nada y aprenderá a predecir puros ceros. Tendríamos un modelo con "99% de accuracy" que nunca salta y pierde siempre.

Escenario P2: ¿Cuántos puntos alcanzará esta partida al morir?

- Variable objetivo (Y): puntuacion_final. Tipo: Numérica continua (Regresión).
- Variables de entrada (X):
    1. rango_jugador (Categórica): Nivel de experiencia del jugador.
    2. fps_promedio (Numérica): Si el dispositivo tiene lag o caídas de frames, el rendimiento baja drásticamente.
    3. intentos_sesion_actual (Numérica): Para medir si el jugador apenas está calentando o ya está fatigado.
    4. high_score_jugador (Numérica): El techo estadístico de esa persona.
    5. hora_dia (Numérica): Factores de luz o cansancio del usuario.
- Granularidad: Una partida completa por fila (desde el inicio hasta el Game Over).

- Tamaño mínimo razonable: Unas 10,000 partidas. Al depender del factor humano, hay mucho ruido estadístico, por lo que necesitamos mucha varianza para trazar una buena regresión.
- Riesgo si está mal definido: Data Leakage (Fuga de datos). Si por error metemos una variable como tiempo_total_jugado en la X, le estaríamos pasando la respuesta al modelo desde el futuro. En entrenamiento se verá perfecto, pero en producción fallará porque no podemos saber ese dato antes de que empiece la partida.

Escenario P3: ¿Qué tipo de obstáculo viene próximo?

- Variable objetivo (Y): proximo_obstaculo. Tipo: Categórica multiclase (ej. 0=Nada, 1=Cactus_S, 2=Cactus_L, 3=Pajaro).
- Variables de entrada (X):
    1. score_actual (Numérica): Crítico, porque las reglas cambian (los pájaros no spawnean al inicio).
    2. obstaculo_anterior (Categórica): Para ver si el motor evita repetir el mismo obstáculo muchas veces seguidas.
    3. distancia_desde_ultimo_spawn (Numérica): Para encontrar los "cooldowns" del juego.
    4. velocidad_actual (Numérica): Ciertos obstáculos podrían agruparse distinto a altas velocidades.
    5. tipo_terreno_previo (Categórica): Lo que acaba de pasar en pantalla.
- Granularidad: Por evento de spawn (una fila se registra solo cuando el motor decide crear un objeto nuevo).
- Tamaño mínimo razonable: Unos 5,000 eventos para capturar la distribución estadística del RNG.
- Riesgo si está mal definido: Granularidad equivocada. Si guardamos esto por frame, tendríamos miles de filas vacías donde el "próximo obstáculo" sigue siendo el mismo. Esto saturaría la RAM con datos redundantes y causaría un sobreajuste terrible.

Misión 2:

1. El patrón en la fila donde died=1 (frame 82) Analizando la tabla, en el frame 80 y 81 el dinosaurio estaba en el aire (jump = 1), esquivando un obstáculo que estaba a 55px y luego a 38px. Sin embargo, en el frame 82 la variable jump pasa a 0 (el dino aterrizó o cortó el salto) y la distancia al obstáculo es de apenas 12px. Básicamente, cayó directo sobre la hitbox del cactus.

2. ¿Es score una buena variable para predecir la muerte inmediata? No, es una pésima variable para el Escenario P1. En la muestra vemos que durante los frames 80, 81 y 82, el score se estanca en el valor 16. Un modelo matemático no puede extraer un patrón de colisión a partir de un valor constante; además, el score está

completamente correlacionado con time_ms y speed. Sirve como contexto general, pero no tiene la resolución espacial para evaluar un choque inminente.

3. Columnas críticas que nos faltan para P1 El esquema actual está muy pobre para detectar colisiones. Faltan:

- posicion_y: Un booleano de jump no es suficiente. Necesitamos saber la altura exacta en píxeles para saber si el dino va ascendiendo o si ya va en picada hacia el cactus.
- is_ducking (Binaria): Indispensable para cuando salen los pájaros, para saber si la hitbox del jugador está reducida.
- ancho_obstaculo: No es lo mismo el tiempo de vuelo requerido para saltar un cactus pequeño que para saltar tres pegados.

4. El problema con la definición actual de died Tal como está en el diccionario ("1 solo en el último frame de la sesión"), died solo sirve para documentar el Game Over, pero no sirve para predecir. Si el modelo evalúa el frame 81, ve died = 0 y piensa que todo está bien. Para que esto funcione en P1, tendríamos que aplicar una operación de desplazamiento (como un shift(-1) en Pandas o con funciones de ventana en SQL) para crear una variable target_next_frame. Así, el frame 81 heredaría el 1 del frame 82, enseñándole al modelo que ese estado en el aire y a 38px es el que desencadena la muerte.

Bloque 2

Misión 3: Las diez preguntas del analista

1. Tres preguntas del checklist aplicadas a nuestro dataset hipotético:

- ¿La clase objetivo está balanceada? (Q3): Definitivamente no. En un juego de reflejos, el 99% del tiempo estás corriendo a salvo y solo un instante chocas. Si no aplicamos técnicas como class weights o submuestreo, el modelo va a ignorar los choques por completo.
- ¿Variables correlacionadas? (Q5): Hipótesis: score y speed van a tener una correlación positiva altísima (casi de 1.0). Conforme pasa el tiempo, ganas puntos y el juego acelera. Meter ambas al mismo tiempo en ciertos modelos nos daría problemas de multicolinealidad, haciendo que el modelo se confunda sobre qué variable realmente está causando la dificultad.
- ¿Outliers? (Q7): Es probable que encontremos valores atípicos en dist_obstacle (ej. distancias negativas si las hitboxes se superponen un par de frames antes de que el motor registre el "Game Over"). Hay que limpiar estos bugs gráficos antes de entrenar para no meterle ruido al algoritmo.

2. El problema de la independencia (i.i.d.) y por qué es un error mezclar frames:

Asumir que cada fila (frame) es independiente es un error garrafal en series de tiempo. El frame 101 es básicamente una copia del frame 100 pero con el dinosaurio movido un par de píxeles. Si mandas el frame 100 a los datos de entrenamiento y el 101 a los de prueba, el modelo no está aprendiendo a jugar; está memorizando. Es vital separar los datos agrupando por la sesión completa (session_id), aislando las partidas enteras, de la misma forma en que estructurarías una llave foránea en una base de datos relacional para no mezclar contextos de distintas entidades.

3. Ejemplo de Data Leakage con score o time_ms:

Si para predecir si vas a chocar en el siguiente tick (P1) metemos como variable el score final que alcanzó la partida, le estamos pasando información del futuro. Sería como intentar desarrollar un modelo predictivo para saber si vas a cerrar una venta hoy, pero usando como variable de entrada la factura cobrada de la semana que viene. El modelo se verá con una precisión perfecta en las pruebas porque ya tiene la respuesta, pero en el mundo real fallará por completo.

Misión 4: Leer distribuciones sin gráfica (Interpretación)

1. Desbalance del problema P1 (cuantificado):

El problema está brutalmente desbalanceado. Según la tabla, tenemos unas 50 partidas que generan en total ~12,000 frames. Como solo se muere una vez por partida, tenemos exactamente 50 filas donde died = 1.

- Cálculo: $50 / 12,000 = 0.0041$
  Esto significa que la clase positiva (muerte) representa apenas el 0.41% del dataset. El 99.59% son puros ceros.

2. Implicación para la métrica a usar:

Usar Accuracy (Exactitud) aquí es inútil. Un modelo tonto que hardcodee un return 0 (nunca salta, siempre dice que está a salvo) tendría un 99.59% de calificación, pero perdería la partida en 10 segundos. Para este tipo de asimetría necesitamos mirar el Recall (de todas las veces que de verdad iba a chocar, ¿cuántas logró detectar a tiempo?) y la Precision (cuando decidió saltar porque "venía un choque", ¿era real o saltó al vacío?). La métrica ideal a optimizar sería el F1-Score, que nos da la media armónica entre ambas.

3. Evaluación de dist_obstacle como predictor:

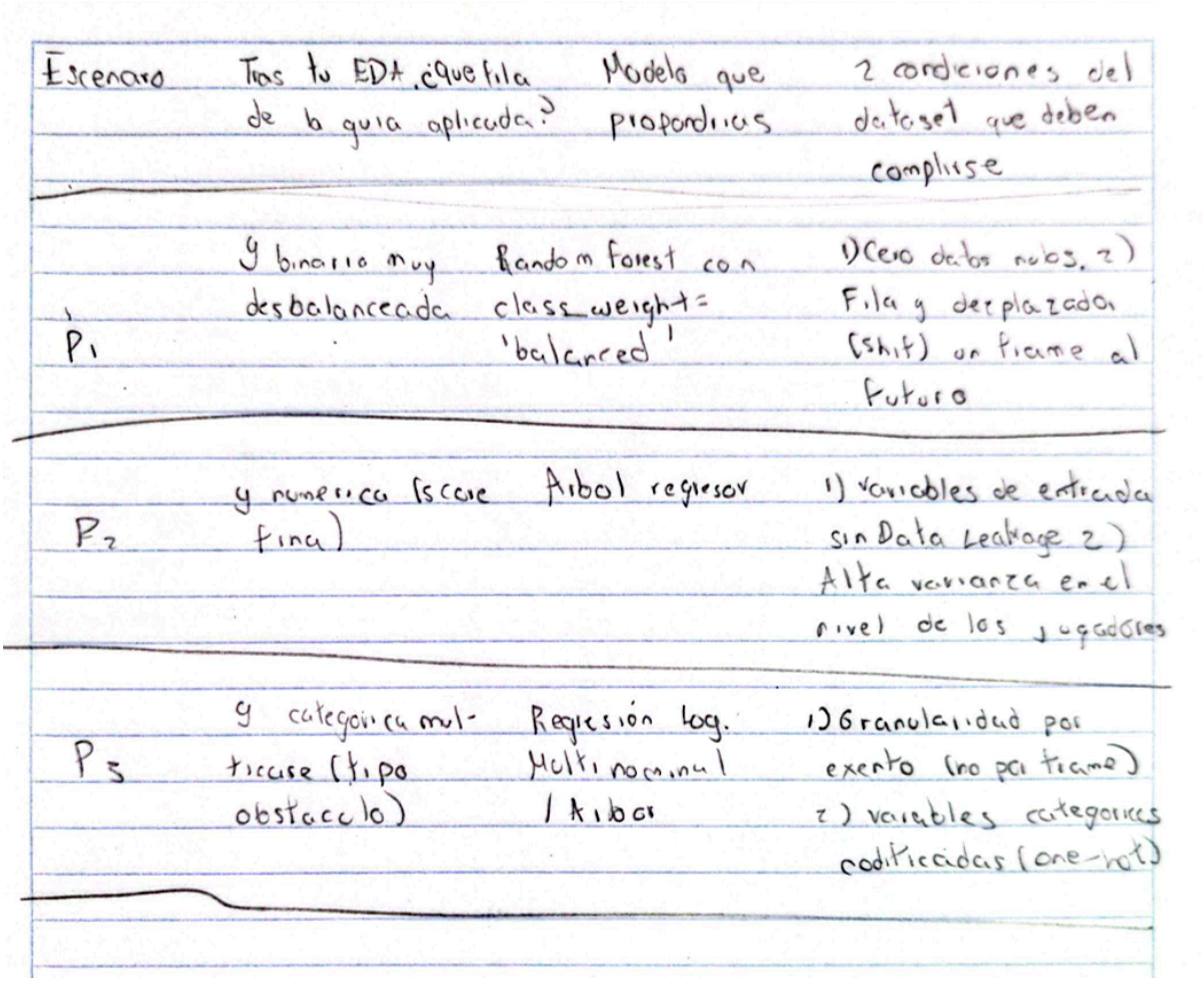
Los datos indican que las muertes se concentran cuando dist_obstacle < 20, mientras que el rango de la variable llega hasta 220. Esto la convierte en un predictor (feature) excelente. Nos está dando un umbral natural clarísimo: funciona como la condición principal de un árbol de decisión (ej. if dist < 20 and jump == 0 -> crash). Separa perfectamente el peligro del estado seguro.

4. Distribución del score y selección de enfoque:

La estadística descriptiva nos grita que hay un sesgo enorme (asimetría positiva).

- La Media (28) es mucho mayor que la Mediana (18).
- Esto y el comentario de "cola larga hacia la derecha" significa que la gran mayoría de las partidas mueren súper rápido (puntos bajos), pero hay unos cuantos registros aislados que llegan hasta 120.
  Una regresión lineal simple fallaría aquí porque asume que los errores se distribuyen de forma normal. Para manejar esta cola larga, necesitaríamos aplicar una transformación matemática primero (como el logaritmo del score) o descartar la regresión simple e irnos por modelos robustos ante valores atípicos y no lineales, como árboles de decisión o un Random Forest.

Bloque 3



Mision 6

Árbol profundo desaconsejado: Imagina que queremos predecir P2 (score final) y tenemos 150 variables (clics por segundo, RAM libre, temperatura del CPU...) pero solo 30 partidas jugadas. Un árbol profundo memorizará esas 30 partidas exactamente (overfitting brutal). El EDA nos diría que N es muy chico para tantas dimensiones.

Cuándo usar Redes Neuronales: Tendría sentido si en lugar de un CSV tabular limpio tuviéramos capturas de pantalla del juego (píxeles crudos). El EDA mostraría que no tenemos columnas numéricas de distancia, justificando una CNN (Red Neuronal Convolucional) para extraer la geometría visual directamente de las imágenes.

Reglas fijas vs. Modelo: Sí, P1 se puede resolver con si dist_obstacle < 15 y jump=0 -> saltar. Ventaja de reglas fijas: Es instantáneo y perfecto si conoces las medidas exactas. Límite: Si el desarrollador lanza un parche que hace al dino más rápido o al cactus más ancho, la regla se rompe. Un modelo de ML, en cambio, se adapta solo con reentrenarlo con los datos del nuevo parche.